Trabajo Práctico: Programación Defensiva y Endurecimiento de Instancias

Equipo 01 – Integrantes

- Tomás Ruiz
- Facundo Sorasio
- Mateo Sparta
- Rodrigo Vargas
- José Zerpa

TP 5: Programación Defensiva contra SQLi

Consigna

Identifique una vulnerabilidad de inyección SQL en un bloque de código que utiliza concatenación de cadenas y reescríbalo utilizando consultas parametrizadas.

Análisis y Resolución

La inyección SQL (SQLi) ocurre cuando un atacante inserta código SQL malicioso en una consulta a través de datos de entrada de la aplicación. Es una de las vulnerabilidades más antiguas y persistentes en aplicaciones web: figura desde hace años en el *OWASP Top 10*, actualmente dentro de la categoría A03:2021 – Injection.

La causa principal es la construcción de consultas mediante concatenación directa de cadenas, que fusiona la lógica del código con la entrada del usuario y rompe la separación que debe existir entre instrucciones SQL y datos.

Código Vulnerable (Concatenación de Cadenas)

El siguiente ejemplo muestra una función hipotética que busca un usuario, donde la entrada del usuario se incorpora directamente a la consulta:

```python
def get_user_data(user_id):
    # La variable 'query' se construye concatenando la entrada del usuario.
    query = "SELECT * FROM Users WHERE id = '" + user_id + "';"
    # Ejecutar query...
    return db_execute(query)
```

Si un atacante introduce la cadena maliciosa `' OR '1'='1' --` en la variable `user_id`, la consulta SQL final que ejecuta la aplicación es:

```sql
SELECT * FROM Users WHERE id = '' OR '1'='1' --';
```

La cláusula `WHERE` se ejecuta normalmente, pero la condición `'1'='1'` se evalúa siempre como verdadera y, combinada con el `OR`, hace que el filtro pierda toda capacidad de discriminar registros. El motor termina devolviendo todas las filas de la tabla `Users`. Los dos guiones finales

(`--`) comentan el resto de la consulta, neutralizando la comilla simple que quedaría suelta y evitando un error de sintaxis.

El sistema podría ser vulnerable a un ataque incluso más destructivo. Si la cadena introducida es:

```sql
'; DROP TABLE Users; --
```

La consulta resultante que se ejecuta en la base de datos sería:

```sql
SELECT * FROM Users WHERE id = ''; DROP TABLE Users; --';
```

En ambos casos, la aplicación entrega al motor una cadena donde el input del usuario modifica la semántica del comando. El segundo vector ilustra que el riesgo no se limita a la filtración de datos: puede derivar en pérdida total de la información.

Solución: Consultas Parametrizadas (Prepared Statements)

Las consultas parametrizadas son la defensa primaria recomendada. La estructura de la consulta se define con marcadores de posición (`?` o `:nombre`), y el valor suministrado por el usuario se envía al motor por separado.

```python
def get_user_data_safe(user_id):
    # La consulta se prepara con un marcador de posición (placeholder: '?').
    sql_template = "SELECT * FROM Users WHERE id = ?;"
    # Los datos del usuario se pasan como un parámetro separado.
    return db_execute_safe(sql_template, [user_id])
```

El punto clave es que, con un prepared statement real, la consulta y los datos viajan por canales separados a nivel de protocolo: el motor primero parsea el template SQL con sus placeholders y recién después recibe los valores como datos. Esos valores nunca se reinterpretan como SQL ejecutable. No se trata de un mecanismo de “escape” automático sino de una separación estructural entre código y datos, que es justamente lo que las hace más seguras que el escape manual de cadenas.

Si el atacante intenta la misma inyección (`' OR '1'='1' --`), el motor interpreta esa cadena como un único valor literal y busca un `id` exactamente igual a ese texto. Como ese ID no existe, la consulta no devuelve resultados y el ataque falla.

Defensa en profundidad

Las consultas parametrizadas son la barrera principal, pero en una arquitectura seria se complementan con:

- Validación y normalización de la entrada del usuario en la capa de aplicación.
- Principio de mínimo privilegio en el usuario de base de datos con el que se conecta la aplicación (que no sea owner de las tablas ni tenga permisos de `DROP` salvo que sea estrictamente necesario).
- Uso de ORMs o capas de acceso a datos configurados para parametrizar por defecto, evitando que un desarrollador descuidado vuelva a concatenar.
- Monitoreo y logging de consultas anómalas.

TP 6: Endurecimiento (Hardening) de Instancias

Consigna

Basándose en los Benchmarks del CIS, elabore una lista de tres acciones críticas para asegurar un motor PostgreSQL recién instalado.

Análisis y Resolución

El proceso de endurecimiento (*hardening*) de un servidor de base de datos como PostgreSQL es una medida de seguridad fundamental que implica configurar el sistema para reducir su superficie de ataque y mitigar riesgos conocidos. Siguiendo las directrices del **CIS PostgreSQL Benchmark** (Center for Internet Security) —en particular las secciones sobre conexiones, autenticación y permisos de archivos— las acciones se centran en la gestión de accesos de red, el principio de mínimo privilegio y la autenticación fuerte. El objetivo es asegurar que un compromiso de la instancia no resulte en un control total del sistema subyacente.

Acciones Críticas de Endurecimiento para PostgreSQL

1. Restringir las Interfaces de Red (listen_addresses)

Detalle técnico (acción): El archivo de configuración principal, `postgresql.conf`, contiene el parámetro `listen_addresses`. Por defecto, una instalación nueva puede establecer este valor como `listen_addresses = '*'`, lo que indica que el servidor escuchará conexiones entrantes en todas las interfaces de red disponibles, incluyendo direcciones públicas.

La acción crítica consiste en modificar esa línea a algo como:

```
CONFIG
listen_addresses = 'localhost, 192.168.1.10'
```

(donde la IP corresponde a la interfaz interna del servidor de aplicaciones).

Implementación de ejemplo:

1. Localizar y editar el archivo `postgresql.conf`.
2. Cambiar la línea de configuración al valor deseado.
3. Reiniciar el servicio de PostgreSQL para aplicar el cambio.

Principio de seguridad aplicado: restricción de interfaces de red y mínima exposición. Al limitar las direcciones de escucha se implementa una defensa en profundidad esencial. Si la instancia solo escucha en `localhost` (127.0.0.1) o en una IP privada específica, se vuelve inaccesible desde fuera de la red de confianza, incluso si el firewall de red falla. Esto reduce significativamente la superficie de ataque y elimina la exposición a ataques de reconocimiento o fuerza bruta originados fuera del segmento de red autorizado.

2. Ejecutar el Servicio con Privilegios Mínimos

Detalle técnico (acción): Por diseño, PostgreSQL crea y se ejecuta bajo una cuenta de sistema dedicada durante la instalación (típicamente el usuario `postgres`). Esta cuenta tiene permisos estrictamente limitados y no se utiliza para iniciar sesión ni para tareas administrativas generales del sistema operativo.

La acción crítica es la verificación y el mantenimiento constante de este estado:

- Asegurar que el proceso del motor de la base de datos **nunca** se ejecute bajo la cuenta `root` ni bajo otra cuenta con privilegios administrativos elevados.
- Los directorios de datos de PostgreSQL deben ser propiedad exclusiva del usuario `postgres` y tener permisos estrictos (típicamente `0700`) que impidan que otros usuarios del sistema puedan leer o modificar archivos de configuración y datos.

Principio de seguridad aplicado: Principio del Mínimo Privilegio (PoLP). Si el proceso del servidor es comprometido por una vulnerabilidad (por ejemplo, un desbordamiento de búfer o una inyección SQL compleja que permita ejecución de código remoto), correr el servicio bajo un usuario no privilegiado como `postgres` garantiza que el atacante herede solo esos privilegios mínimos. Esto impide que escale para instalar rootkits o acceder a archivos sensibles de otros servicios del sistema operativo. Es un aislamiento crítico entre la base de datos y el sistema host.

3. Configurar la Autenticación Estricta del Cliente (pg_hba.conf)

Detalle técnico (acción): El archivo `pg_hba.conf` (Host-Based Authentication) es el principal mecanismo de control de acceso de PostgreSQL. Define qué usuarios, desde qué hosts y a qué bases de datos pueden conectarse, y qué método de autenticación deben usar.

La acción crítica de endurecimiento implica:

1. **Eliminar métodos débiles:** nunca permitir el método `trust`, que admite conexiones sin contraseña.
2. **Imponer cifrado y hashing fuerte:** para conexiones locales (sockets Unix o `localhost`), usar `peer` o `scram-sha-256`. Para conexiones remotas (`host`), imponer estrictamente `scram-sha-256` en lugar de `md5` y requerir conexiones cifradas mediante SSL/TLS. Conviene aclarar que `md5` no está formalmente eliminado de PostgreSQL, pero está desaconsejado: desde PostgreSQL 14 el método por defecto es `scram-sha-256`, y `md5` es vulnerable a ataques de *pass-the-hash* porque el propio hash funciona como credencial.
3. **Restricción del superusuario:** el acceso al superusuario `postgres` (o cualquier rol con atributos `SUPERUSER`) debe restringirse a `localhost` o a direcciones IP de administración altamente controladas.

Ejemplo de entradas endurecidas en pg_hba.conf:

```
CONFIG
# Conexiones locales por socket Unix: peer mapea el usuario del SO
local   all   all                             peer

# Resto de conexiones: scram-sha-256 obligatorio, nunca trust ni md5
host    all   all        192.168.1.0/24       scram-sha-256

# Superusuario restringido a localhost con autenticación fuerte
host    all   postgres   127.0.0.1/32         scram-sha-256
```

Principio de seguridad aplicado: gestión de accesos y autenticación fuerte. Esta configuración asegura que solo las entidades autorizadas puedan conectarse y elimina el riesgo de ataques de *pass-the-hash* o de escuchas de credenciales en la red. El uso de `scram-sha-256` garantiza que las contraseñas no se almacenen ni se transmitan de forma insegura, cumpliendo con los estándares modernos. Y al restringir las capacidades del superusuario se limita el daño potencial si esas credenciales son robadas.